

Course: Data Structures (CSA03) | CO1 | Assessment Tool 2

Data Structures: Technical Presentation

Array | Graph | Stack | Hash Table



Name: M.Dinesh kumar

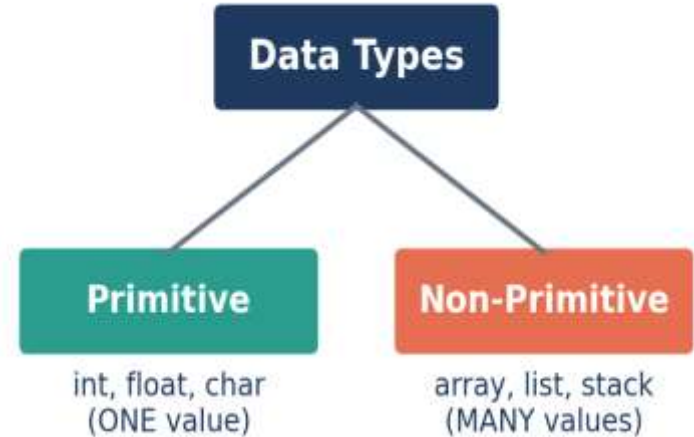
Register No:192525298

Q1: Problem & Data Types

2

Storing Marks of 1000 Students

- 1 Store and manage marks of 1000 students
- 2 Primitive types (int, float) hold only one value
- 3 1000 separate variables is impractical
- 4 Non-primitive types (array, list) hold many values
- 5 So a non-primitive structure is needed



Q2: Why Choose an Array?

3

Storing Marks of 1000 Students

- 1 Size is known: fixed at 1000 students
- 2 Marks stored in continuous memory
- 3 Direct access by index, e.g. `marks[i]`
- 4 Easy to traverse, update and sort
- 5 Simple code, no extra pointer memory

`marks[ ]` array: continuous memory

85	72	90	66	78	...	94
0	1	2	3	4	...	999

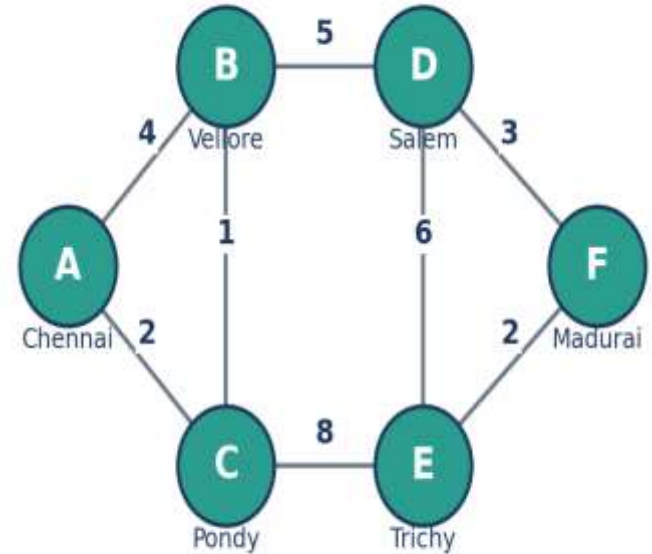
index →

Q2: Problem & Need

5

Navigation System: City Connections & Shortest Route

- 1 Navigation stores cities and road connections
- 2 Every road has a distance or travel time
- 3 We must find the shortest route
- 4 Arrays and lists cannot show connections
- 5 So a non-linear structure is needed

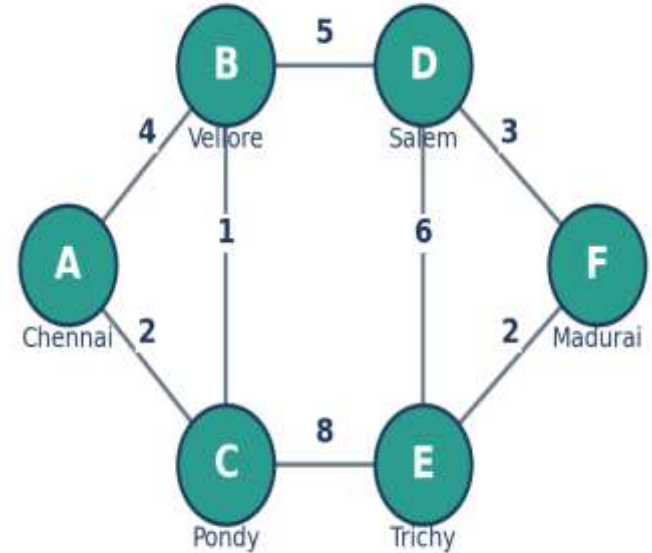


Q2: Finding Shortest Route

7

Navigation System: City Connections & Shortest Route

- 1 Dijkstra's algorithm finds the shortest path
- 2 Starts at source, picks nearest city
- 3 Updates distances of neighbour cities
- 4 Time: $O(E \log V)$ with priority queue
- 5 Used in real map applications



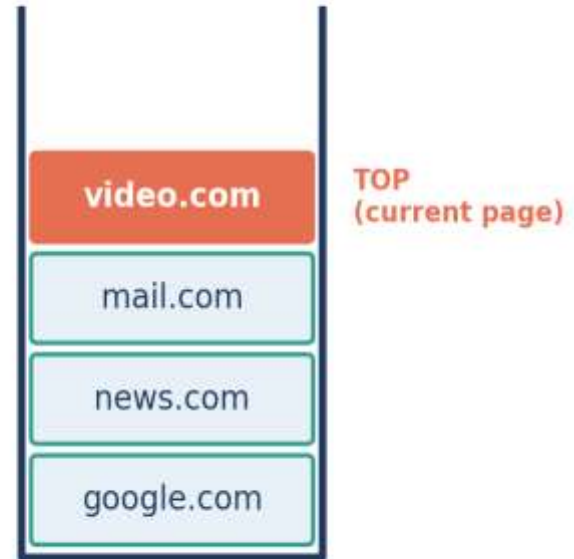
Q3: Problem & Need

8

Web Browser Back Navigation

- 1 Browser must go back to earlier pages
- 2 Last visited page should return first
- 3 This behaviour is called LIFO
- 4 We need an ADT that follows LIFO
- 5 Stack ADT is the suitable choice

Pages visited (stack)



Q3: Stack ADT Operations

9

Web Browser Back Navigation

- 1 `push()`: add the new page on top
- 2 `pop()`: remove top page (Back button)
- 3 `peek()`: view the current page
- 4 `isEmpty()`: check if any page is left
- 5 Only the top end is used



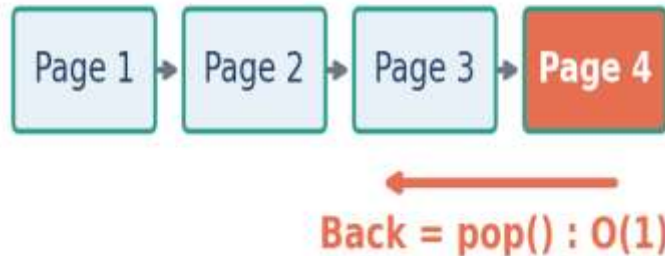
Q3: Performance Efficiency

10

Web Browser Back Navigation

- 1 Push and pop take $O(1)$ time
- 2 No searching or shifting of pages
- 3 Memory used is $O(n)$ for n pages
- 4 Simple and fast to implement
- 5 Back button responds instantly

Forward visits = push, Back = pop

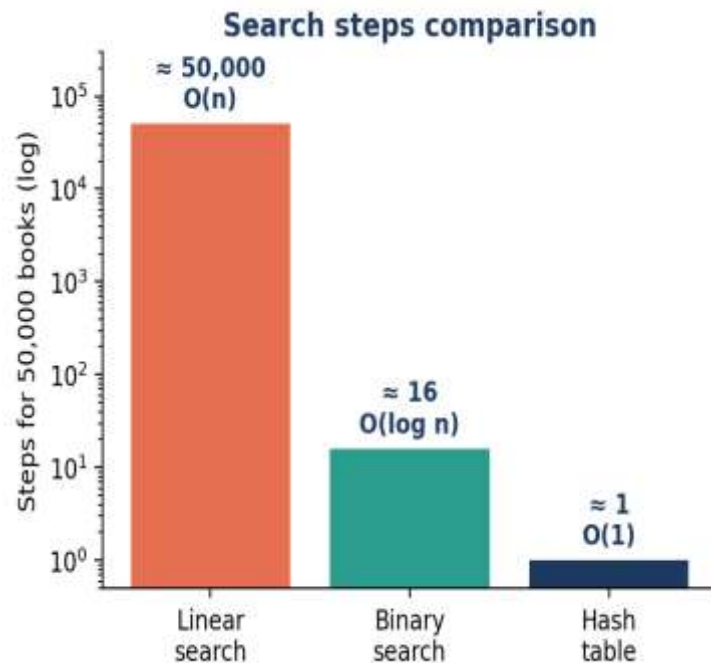


Q4: Problem & Need

11

Searching 50,000 Books by ISBN

- 1 Library has 50,000 book records
- 2 Every book has a unique ISBN number
- 3 Search must be very fast
- 4 Linear search may check all 50,000
- 5 So a faster structure is needed

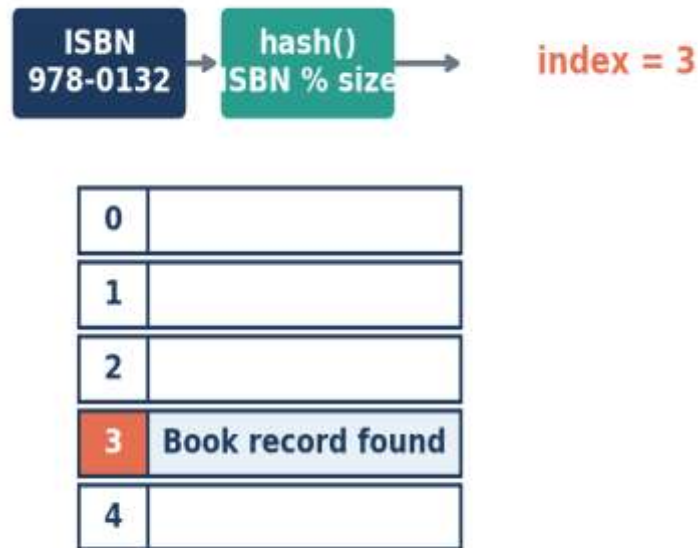


Q4: Why Hash Table?

12

Searching 50,000 Books by ISBN

- 1 Hash function turns ISBN into an index
- 2 Index points straight to the book record
- 3 Average search time is $O(1)$
- 4 Collisions handled by chaining
- 5 Insert and delete are also fast



Q4: Time Complexity Comparison

13

Searching 50,000 Books by ISBN

- 1 Linear search: $O(n)$, about 50,000 steps
- 2 Binary search: $O(\log n)$, about 16 steps
- 3 Binary search needs sorted data
- 4 Hash table: $O(1)$, about 1 step
- 5 Conclusion: hash table is the best choice

